Material de Estudo: Persistência de Objetos em Java

Disciplina de Programação Orientada a Objetos

1 Introdução à Persistência de Objetos

Em Programação Orientada a Objetos (POO), os objetos existem na memória RAM durante a execução do programa. Quando o programa é encerrado, todos os dados são perdidos. A **persistência** é o mecanismo que permite salvar o estado desses objetos em um meio de armazenamento não volátil (como um disco rígido), para que possam ser recuperados e utilizados posteriormente, mesmo após o encerramento da aplicação.

Neste material, abordaremos três níveis de persistência em Java, com forte ênfase no tratamento das exceções geradas por operações de Entrada e Saída (I/O) e comunicação com banco de dados.

2 Persistência Nativa em Java (Serialização)

A forma mais fundamental de persistência em Java é a **Serialização**. Trata-se do processo de converter o estado de um objeto em um fluxo de bytes (*byte stream*). Esse fluxo pode ser gravado em um arquivo físico. O processo inverso, de ler o fluxo de bytes e recriar o objeto na memória, é chamado de **Desserialização**.

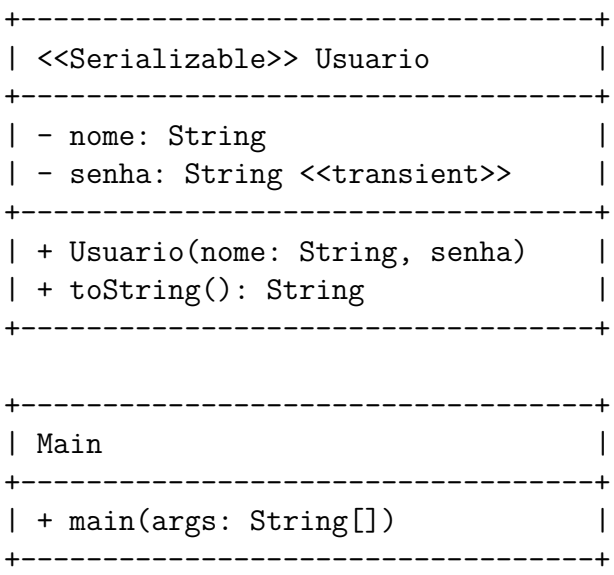
2.1 Teoria e Conceitos Principais

Para que um objeto possa ser serializado em Java, sua classe deve implementar a interface marcadora `java.io.Serializable`. Esta interface não possui métodos; ela serve apenas para sinalizar à Máquina Virtual Java (JVM) que os objetos daquela classe podem ser convertidos em bytes.

- **ObjectOutputStream:** Classe responsável por gravar objetos no fluxo de saída (ex: arquivo).
- **ObjectInputStream:** Classe responsável por ler os objetos do fluxo de entrada.
- **transient:** Modificador utilizado em atributos que **não** devem ser serializados (como senhas ou dados temporários).
- **Exceções Comuns:** `IOException` (falhas de leitura/escrita) e `ClassNotFoundException` (classe do objeto lido não encontrada no projeto).

2.2 Diagrama de Classes

Abaixo temos um rascunho da representação estrutural do nosso exemplo prático, possuindo uma classe de modelo (`Usuario`) e a classe principal (`Main`).



2.3 Exemplo Prático

O código abaixo pode ser executado em um único arquivo. Note o uso do bloco `try-with-resources` para garantir o fechamento dos fluxos e o tratamento de exceções.

```java
import java.io.*;

// Classe modelo que sera persistida
class Usuario implements Serializable {
    private static final long serialVersionUID = 1L;
    private String nome;
    private transient String senha; // Nao sera salva no arquivo

    public Usuario(String nome, String senha) {
        this.nome = nome;
        this.senha = senha;
    }

    @Override
    public String toString() {
        return "Usuario [Nome=" + nome + ", Senha=" + senha + "]";
    }
}

// Classe Principal
public class Main {
    public static void main(String[] args) {
        Usuario user = new Usuario("Dani", "123456");
        String arquivo = "usuario.dat";

        // 1. Serializacao (Gravacao)
        try (ObjectOutputStream oos = new ObjectOutputStream(new
            FileOutputStream(arquivo))) {
             oos.writeObject(user);
```

```
            System.out.println("Objeto serializado com sucesso!");
        } catch (IOException e) {
            System.err.println("Erro ao gravar o arquivo: " + e.
                getMessage());
        }

        // 2. Desserializacao (Leitura)
        try (ObjectInputStream ois = new ObjectInputStream(new
            FileInputStream(arquivo))) {
            Usuario userLido = (Usuario) ois.readObject();
            System.out.println("Objeto recuperado: " + userLido);
            // A senha sera impressa como 'null' devido ao modificador
                transient
        } catch (IOException | ClassNotFoundException e) {
            System.err.println("Erro ao ler o objeto: " + e.getMessage()
                );
        }
    }
}
```

Listing 1: Exemplo de Serialização e Desserialização

2.4 Exercícios Resolvidos (Persistência Nativa)

Nível Fácil

1. **Exercício 1:** Crie um programa que serialize um objeto `Produto` (nome, preco). Trate especificamente a `FileNotFoundException`.

```
import java.io.*;

class Produto implements Serializable {
    String nome; double preco;
    Produto(String n, double p) { nome = n; preco = p; }
}

public class Main {
    public static void main(String[] args) {
        try (ObjectOutputStream oos = new ObjectOutputStream(new
            FileOutputStream("prod.dat"))) {
            oos.writeObject(new Produto("Notebook", 3500.0));
        } catch (FileNotFoundException e) {
            System.err.println("Caminho nao encontrado: " + e.
                getMessage());
        } catch (IOException e) {
            System.err.println("Erro de I/O: " + e.getMessage());
        }
    }
}
```

2. **Exercício 2:** Leia o objeto do Exercício 1 e trate a exceção `ClassNotFoundException`.

```
import java.io.*;

class Produto implements Serializable { String nome; double preco;
    }

```

```java
public class Main {
    public static void main(String[] args) {
        try (ObjectInputStream ois = new ObjectInputStream(new
            FileInputStream("prod.dat"))) {
            Produto p = (Produto) ois.readObject();
            System.out.println("Lido: " + p.nome);
        } catch (ClassNotFoundException e) {
            System.err.println("Classe Produto nao existe no
                projeto atual: " + e.getMessage());
        } catch (IOException e) {
            System.err.println("Erro de leitura: " + e.getMessage()
                );
        }
    }
}
```

Nível Médio

3. **Exercício 3:** Crie uma classe `Configuracao` com um atributo `chaveAPI` marcado como `transient`. Salve e recupere o objeto, comprovando que a chave não foi salva.

```java
import java.io.*;

class Configuracao implements Serializable {
    String url;
    transient String chaveAPI;
    Configuracao(String u, String c) { url = u; chaveAPI = c; }
}

public class Main {
    public static void main(String[] args) {
        try {
            ObjectOutputStream oos = new ObjectOutputStream(new
                FileOutputStream("config.dat"));
            oos.writeObject(new Configuracao("http://api.com", "ABC
                -123"));
            oos.close();

            ObjectInputStream ois = new ObjectInputStream(new
                FileInputStream("config.dat"));
            Configuracao c = (Configuracao) ois.readObject();
            ois.close();

            System.out.println("URL: " + c.url + " | Chave: " + c.
                chaveAPI); // chaveAPI sera null
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

4. **Exercício 4:** Crie um programa que grave uma `ArrayList` de objetos `Cliente` em um arquivo. Trate `IOException`.

```java
import java.io.*;
import java.util.*;

```

```java
class Cliente implements Serializable { String nome; Cliente(String n) { nome = n; } }

public class Main {
    public static void main(String[] args) {
        List<Cliente> lista = new ArrayList<>();
        lista.add(new Cliente("Ana"));
        lista.add(new Cliente("Carlos"));

        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("clientes.dat"))) {
            oos.writeObject(lista); // ArrayList eh serializavel
            System.out.println("Lista salva.");
        } catch (IOException e) {
            System.err.println("Erro: " + e.getMessage());
        }
    }
}
```

Nível Difícil

5. **Exercício 5:** Modifique um fluxo para gravar múltiplos objetos individualmente e use a exceção `EOFException` (End Of File) como controle para parar a leitura.

```java
import java.io.*;

class Log implements Serializable { String msg; Log(String m) { msg = m; } }

public class Main {
    public static void main(String[] args) throws Exception {
        ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("logs.dat"));
        oos.writeObject(new Log("Erro 1"));
        oos.writeObject(new Log("Erro 2"));
        oos.close();

        try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream("logs.dat"))) {
            while (true) { // Loop infinito intencional
                Log log = (Log) ois.readObject();
                System.out.println("Log lido: " + log.msg);
            }
        } catch (EOFException e) {
            System.out.println("Fim do arquivo alcancado em seguranca.");
        } catch (IOException | ClassNotFoundException e) {
            System.err.println("Erro critico: " + e.getMessage());
        }
    }
}
```

6. **Exercício 6:** Implemente métodos customizados `writeObject` e `readObject` na classe para criptografar uma string simples (invertendo-a) antes de serializar, descriptografando ao ler.

```java
import java.io.*;
```

```
2
3  class Segredo implements Serializable {
4      private String texto;
5      Segredo(String t) { texto = t; }
6
7      private void writeObject(ObjectOutputStream oos) throws
           IOException {
8          String invertido = new StringBuilder(texto).reverse().
               toString();
9          oos.writeObject(invertido);
10     }
11
12     private void readObject(ObjectInputStream ois) throws
           IOException, ClassNotFoundException {
13         String lido = (String) ois.readObject();
14         this.texto = new StringBuilder(lido).reverse().toString();
15     }
16     public String getTexto() { return texto; }
17 }
18
19 public class Main {
20     public static void main(String[] args) throws Exception {
21         Segredo s = new Segredo("MENSAGEM SECRETA");
22         ObjectOutputStream oos = new ObjectOutputStream(new
               FileOutputStream("sec.dat"));
23         oos.writeObject(s); oos.close();
24
25         ObjectInputStream ois = new ObjectInputStream(new
               FileInputStream("sec.dat"));
26         Segredo s2 = (Segredo) ois.readObject(); ois.close();
27         System.out.println("Recuperado: " + s2.getTexto());
28     }
29 }
```

3 Introdução a Banco de Dados Relacional e JDBC com SQLite

Embora a serialização seja útil, sistemas reais requerem consultas complexas, concorrência e integridade. Para isso, utilizamos Bancos de Dados Relacionais (SGBDR). Em um banco de dados relacional, os dados são organizados em **Tabelas** com Linhas (registros) e Colunas (atributos). A linguagem padrão para manipular esses dados é o SQL (*Structured Query Language*).

O **JDBC (Java Database Connectivity)** é a API oficial do Java para executar instruções SQL. O **SQLite** é um SGBDR leve que armazena todo o banco em um único arquivo local, dispensando a instalação de servidores complexos.

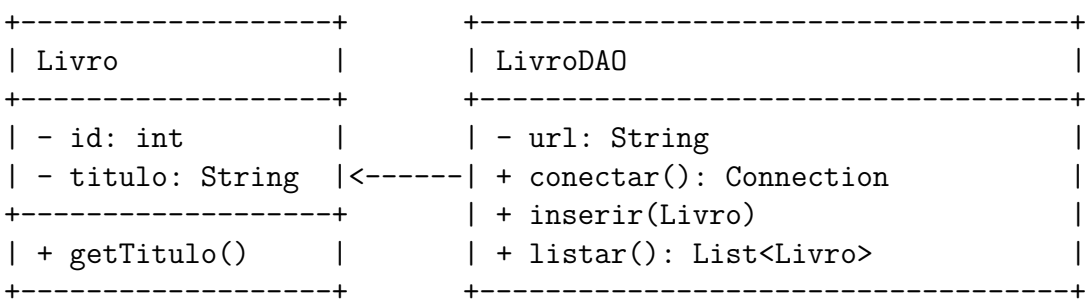
3.1 Teoria e Tratamento de Exceções

Para usar o JDBC, precisamos dos seguintes componentes:

- `Connection`: Mantém a sessão ativa com o banco.
- `PreparedStatement`: Prepara e executa instruções SQL, protegendo contra *SQL Injection*.
- `ResultSet`: Armazena os resultados de uma consulta (`SELECT`).
- **SQLException:** A exceção principal e obrigatória do JDBC. Qualquer erro de sintaxe SQL, falha de conexão ou restrição violada lançará esta exceção.

3.2 Diagrama de Classes

Utilizamos o padrão **DAO** (Data Access Object) para separar a lógica do banco de dados do modelo de negócios.



3.3 Exemplo Prático

Para testar, seria necessário adicionar o arquivo `sqlite-jdbc.jar` ao *classpath* (ou usar ferramentas como GDB Online se tiverem suporte configurado).

```
1 import java.sql.*;
2 import java.util.*;
3
4 // Modelo
5 class Livro {
6     private String titulo;
```

```java
    public Livro(String titulo) { this.titulo = titulo; }
    public String getTitulo() { return titulo; }
}

// Data Access Object
class LivroDAO {
    // Banco local em arquivo
    private String url = "jdbc:sqlite:biblioteca.db";

    public void criarTabela() {
        String sql = "CREATE TABLE IF NOT EXISTS livros (id INTEGER
            PRIMARY KEY, titulo TEXT NOT NULL);";
        try (Connection conn = DriverManager.getConnection(url);
             Statement stmt = conn.createStatement()) {
            stmt.execute(sql);
        } catch (SQLException e) {
            System.err.println("Erro ao criar tabela: " + e.getMessage()
                );
        }
    }

    public void inserir(Livro livro) {
        String sql = "INSERT INTO livros(titulo) VALUES(?)";
        try (Connection conn = DriverManager.getConnection(url);
             PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setString(1, livro.getTitulo());
            pstmt.executeUpdate();
            System.out.println("Livro salvo!");

        } catch (SQLException e) {
            System.err.println("Erro ao inserir: " + e.getMessage());
        }
    }
}

// Classe Principal
public class Main {
    public static void main(String[] args) {
        LivroDAO dao = new LivroDAO();
        dao.criarTabela();
        dao.inserir(new Livro("Programacao Java OO"));
    }
}
```

Listing 2: Conexão, Criação e Inserção com SQLite e JDBC

3.4 Exercícios Resolvidos (JDBC com SQLite)

Nível Fácil

1. **Exercício 1:** Crie um programa que apenas tente se conectar ao banco e capture a `SQLException`.

```java
import java.sql.*;

public class Main {
```

```
    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection("jdbc:
            sqlite:teste.db")) {
            System.out.println("Conexao estabelecida com sucesso.")
                ;
        } catch (SQLException e) {
            System.err.println("Falha na conexao: " + e.getSQLState
                () + " - " + e.getMessage());
        }
    }
}
```

2. **Exercício 2:** Escreva um método para apagar uma tabela ("DROP TABLE IF EXISTS") tratando exceções.

```
import java.sql.*;

public class Main {
    public static void main(String[] args) {
        String url = "jdbc:sqlite:teste.db";
        try (Connection conn = DriverManager.getConnection(url);
             Statement stmt = conn.createStatement()) {
            stmt.execute("DROP TABLE IF EXISTS alunos");
            System.out.println("Tabela apagada com seguranca.");
        } catch (SQLException e) {
            System.err.println("Erro SQL: " + e.getMessage());
        }
    }
}
```

Nível Médio

3. **Exercício 3:** Realize um `SELECT` e itere sobre o `ResultSet`, fechando os recursos adequadamente.

```
import java.sql.*;

public class Main {
    public static void main(String[] args) {
        String url = "jdbc:sqlite:teste.db";
        String sql = "SELECT * FROM sqlite_master"; // Consulta
            generica do SQLite
        try (Connection conn = DriverManager.getConnection(url);
             PreparedStatement pstmt = conn.prepareStatement(sql);
             ResultSet rs = pstmt.executeQuery()) {

            while (rs.next()) {
                System.out.println("Tabela encontrada: " + rs.
                    getString("name"));
            }
        } catch (SQLException e) {
            System.err.println("Erro na consulta: " + e.getMessage
                ());
        }
    }
}
```

4. **Exercício 4:** Crie um código para atualizar o nome de um registro baseando-se no ID usando `PreparedStatement`.

```java
import java.sql.*;

public class Main {
    public static void main(String[] args) {
        String url = "jdbc:sqlite:teste.db";
        String sql = "UPDATE funcionarios SET nome = ? WHERE id = ?
            ";

        try (Connection conn = DriverManager.getConnection(url);
             PreparedStatement pstmt = conn.prepareStatement(sql))
                {

            pstmt.setString(1, "Novo Nome");
            pstmt.setInt(2, 1);
            int afetados = pstmt.executeUpdate();
            System.out.println("Registros alterados: " + afetados);

        } catch (SQLException e) {
            System.err.println("Falha ao atualizar: " + e.
               getMessage());
        }
    }
}
```

Nível Difícil

5. **Exercício 5:** Implemente uma Transação: insira dois registros, mas force um erro no segundo. Realize o `rollback()` no `catch`.

```java
import java.sql.*;

public class Main {
    public static void main(String[] args) {
        String url = "jdbc:sqlite:teste.db";
        Connection conn = null;
        try {
            conn = DriverManager.getConnection(url);
            conn.setAutoCommit(false); // Inicia transacao

            Statement stmt = conn.createStatement();
            stmt.executeUpdate("CREATE TABLE IF NOT EXISTS demo (id
                INTEGER PRIMARY KEY, valor TEXT UNIQUE)");
            stmt.executeUpdate("INSERT INTO demo(valor) VALUES ('A
               ')");
            stmt.executeUpdate("INSERT INTO demo(valor) VALUES ('A
               ')"); // Vai gerar erro UNIQUE

            conn.commit();
        } catch (SQLException e) {
            System.err.println("Erro detectado. Desfazendo
               operacoes: " + e.getMessage());
            if (conn != null) {
                try { conn.rollback(); } catch (SQLException ex) {
                   }
            }
```

```
        } finally {
            if (conn != null) {
                try { conn.close(); } catch (SQLException ex) { }
            }
        }
    }
}
```

6. **Exercício 6:** Crie um método que apague um registro (`DELETE`) e lance uma exceção customizada `IdNaoEncontradoException` se nenhum registro for afetado.

```
import java.sql.*;

class IdNaoEncontradoException extends Exception {
    public IdNaoEncontradoException(String msg) { super(msg); }
}

public class Main {
    public static void main(String[] args) {
        String url = "jdbc:sqlite:teste.db";
        int idAlvo = 999;

        try (Connection conn = DriverManager.getConnection(url);
             PreparedStatement pstmt = conn.prepareStatement("
                DELETE FROM tabela_inexistente WHERE id = ?")) {

            pstmt.setInt(1, idAlvo);
            int linhas = pstmt.executeUpdate();

            if (linhas == 0) {
                throw new IdNaoEncontradoException("Nenhum registro
                    com ID " + idAlvo);
            }
        } catch (SQLException e) {
            System.err.println("Erro SQL: " + e.getMessage());
        } catch (IdNaoEncontradoException e) {
            System.err.println("Erro de Regra de Negocio: " + e.
                getMessage());
        }
    }
}
```

4 JPA (Java Persistence API) com Hibernate

O **JPA** é a especificação oficial do Java para **Mapeamento Objeto-Relacional (ORM)**. O ORM resolve o problema de converter classes Java diretamente em Tabelas de Banco de Dados de maneira transparente. O **Hibernate** é a implementação mais popular e o padrão atual de mercado para o JPA.

Com JPA, nós não escrevemos mais consultas SQL manuais para operações de CRUD (Create, Read, Update, Delete). Manipulamos instâncias de objetos normais (Entidades) e a própria API se encarrega de gerar e executar o SQL correspondente.

4.1 Teoria e Exceções

- **Entidades (@Entity):** Classes Java marcadas com anotações que representam uma tabela.
- **EntityManager:** A interface principal responsável por interagir com o contexto de persistência (salvar, buscar, apagar objetos).
- **PersistenceException:** A classe base para exceções geradas pelo JPA quando há conflitos de infraestrutura ou violação de constraints.
- **IllegalArgumentException:** Lançada pelo EntityManager caso tente-se persistir um objeto nulo ou um tipo que não é uma entidade válida.

4.2 Diagrama de Classes

O mapeamento altera a estrutura da classe adicionando metadados via anotações.

```
+-----------------------------------+
| <<Entity>> Cliente                |
+-----------------------------------+
| @Id @GeneratedValue               |
| - id: Long                        |
| @Column(nullable = false)         |
| - nome: String                    |
+-----------------------------------+
| + Cliente(nome: String)           |
| + getId() : Long                  |
+-----------------------------------+
```

4.3 Exemplo Prático

Nota: Na prática, JPA exige a configuração de um arquivo XML (`persistence.xml`) ou configuração via código. O exemplo demonstra a lógica das classes.

```
1  import javax.persistence.*;
2
3  // Modelo Mapeado
4  @Entity
5  class Cliente {
6      @Id
```

```java
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String nome;

    public Cliente() {} // JPA exige construtor vazio
    public Cliente(String nome) { this.nome = nome; }

    public String getNome() { return nome; }
}

// Classe Principal
public class Main {
    public static void main(String[] args) {
        // Criacao do gerenciador (Baseado no nome definido no
           persistence.xml)
        EntityManagerFactory emf = null;
        EntityManager em = null;

        try {
            emf = Persistence.createEntityManagerFactory("
               unidade_persist");
            em = emf.createEntityManager();

            Cliente cli = new Cliente("Empresa XYZ");

            // Iniciando transacao
            em.getTransaction().begin();
            em.persist(cli); // Salva no banco (Equivalente ao INSERT)
            em.getTransaction().commit();

            System.out.println("Cliente salvo com sucesso!");

        } catch (PersistenceException e) {
            System.err.println("Erro na infraestrutura do Hibernate: " +
                e.getMessage());
            if (em != null && em.getTransaction().isActive()) {
                em.getTransaction().rollback();
            }
        } finally {
            if (em != null) em.close();
            if (emf != null) emf.close();
        }
    }
}
```

Listing 3: Operações básicas com JPA/Hibernate

4.4 Exercícios Resolvidos (JPA/Hibernate)

Nível Fácil

1. **Exercício 1:** Capture uma exceção ao tentar criar um `EntityManagerFactory` com um nome de unidade inexistente.

```java
import javax.persistence.*;
```

```
2
3  public class Main {
4      public static void main(String[] args) {
5          try {
6              EntityManagerFactory emf = Persistence.
                   createEntityManagerFactory("unidade_falsa");
7          } catch (PersistenceException e) {
8              System.err.println("Unidade de persistencia nao
                   encontrada: " + e.getMessage());
9          }
10     }
11 }
```

2. **Exercício 2:** Tente persistir um objeto `null` utilizando `em.persist()` e trate a `IllegalArgumentException`.

```
1  import javax.persistence.*;
2
3  public class Main {
4      public static void main(String[] args) {
5          // Assumindo EntityManager 'em' previamente instanciado e
               mockado para o exercicio
6          EntityManager em = null; // em ambiente real, estaria
               instanciado
7          try {
8              if(em != null) {
9                  em.getTransaction().begin();
10                 em.persist(null);
11                 em.getTransaction().commit();
12             }
13         } catch (IllegalArgumentException e) {
14             System.err.println("O objeto enviado para persistencia
                   nao pode ser nulo: " + e.getMessage());
15         }
16     }
17 }
```

Nível Médio

3. **Exercício 3:** Busque um `Cliente` pelo seu ID utilizando `em.find()`. Caso não exista, imprima um aviso (o JPA retorna null, não gera exceção para find normal).

```
1  import javax.persistence.*;
2
3  public class Main {
4      public static void main(String[] args) {
5          EntityManagerFactory emf = Persistence.
               createEntityManagerFactory("unidade");
6          EntityManager em = emf.createEntityManager();
7
8          try {
9              Long idBuscado = 50L;
10             // Assumindo que a classe Cliente existe
11             Object cliente = em.find(Object.class, idBuscado);
12
13             if (cliente == null) {
14                 System.out.println("Atencao: Cliente ID " +
                       idBuscado + " nao encontrado no banco.");
```

```
            } else {
                System.out.println("Cliente encontrado!");
            }
        } finally {
            em.close();
            emf.close();
        }
    }
}
```

4. **Exercício 4:** Escreva o bloco padrão (try-catch-finally) de transação para atualizar o nome de uma Entidade, garantindo o `rollback` se houver falha.

```
import javax.persistence.*;

public class Main {
    public static void main(String[] args) {
        EntityManagerFactory emf = Persistence.
            createEntityManagerFactory("unidade");
        EntityManager em = emf.createEntityManager();
        EntityTransaction tx = em.getTransaction();

        try {
            tx.begin();
            // Para atualizar no JPA, basta buscar o objeto e
                modificar seus atributos.
            // O commit sincroniza as mudancas (UPDATE automatico).
            Object obj = em.find(Object.class, 1L);
            if (obj != null) {
                // obj.setNome("Novo Nome"); (Depende do cast para
                    a Entidade real)
            }
            tx.commit();
        } catch (Exception e) {
            if (tx != null && tx.isActive()) {
                tx.rollback();
                System.err.println("Transacao revertida: " + e.
                    getMessage());
            }
        } finally {
            em.close(); emf.close();
        }
    }
}
```

Nível Difícil

5. **Exercício 5:** Execute uma consulta JPQL (Java Persistence Query Language) utilizando `em.createQuery()` cujo sintaxe seja inválida, e trate a `IllegalArgumentException` disparada pela construção da query.

```
import javax.persistence.*;

public class Main {
    public static void main(String[] args) {
        EntityManagerFactory emf = Persistence.
            createEntityManagerFactory("unidade");
```

```
        EntityManager em = emf.createEntityManager();

        try {
            // JPQL incorreta intencionalmente (tabela sem
               mapeamento ou erro de sintaxe)
            String jpqlFalha = "SELECIONE c DE Cliente c";
            Query q = em.createQuery(jpqlFalha);
            q.getResultList();
        } catch (IllegalArgumentException e) {
            System.err.println("Erro na sintaxe da JPQL: " + e.
               getMessage());
        } catch (PersistenceException e) {
            System.err.println("Erro na execucao no banco: " + e.
               getMessage());
        } finally {
            em.close(); emf.close();
        }
    }
}
```

6. **Exercício 6:** Tente remover um objeto que foi criado com `new` (estado "Transient"no JPA) usando o `em.remove()`. Trate a `IllegalArgumentException` demonstrando que entidades precisam estar atachadas para serem removidas.

```
import javax.persistence.*;

@Entity class Carro { @Id Long id; }

public class Main {
    public static void main(String[] args) {
        EntityManagerFactory emf = Persistence.
           createEntityManagerFactory("unidade");
        EntityManager em = emf.createEntityManager();

        try {
            em.getTransaction().begin();

            Carro carroNaoAtachado = new Carro();
            // Erro: O objeto nao esta gerenciado pelo
               EntityManager
            em.remove(carroNaoAtachado);

            em.getTransaction().commit();
        } catch (IllegalArgumentException e) {
            System.err.println("Erro ao deletar: Objeto nao eh
               gerenciado (detached/transient). " + e.getMessage())
               ;
            em.getTransaction().rollback();
        } finally {
            em.close(); emf.close();
        }
    }
}
```